

@April 8, 2026 4:02 PM

Summary

Action Items & Next Steps

- ☐ Discard the current complex architecture and create a simplified design ▶ ▶
- ☐ Create an end-user level UI mockup showing how the interface will look ▶ ▶
- ☐ Redesign the system to query existing bug tracking systems (Jira, Bugzilla, GitHub) rather than replicating their data ▶ ▶ ▶
- ☐ Remove the unified bug submission portal concept - users should continue using their existing systems ▶ ▶
- ☐ Study modern agentic AI technologies including agent [skills.md](#) and MCP (Model Context Protocol) ▶
- ☐ Create a high-level design showing how to gather information from multiple data sources using agentic AI ▶
- ☐ Document design goals and requirements based on the revised understanding ▶
- ☐ Share updated design in chat for feedback when ready ▶

Problem Statement Presented

- HPE deals with complex hybrid cloud systems including storage, compute, networking, and software ▶
- Different engineering teams use different bug tracking systems: Jira, Bugzilla, and GitHub ▶
- A single bug can involve multiple components across different systems ▶
- Key pain points identified: ▶
 - Data is scattered across disconnected systems
 - No unified view for customers and executives
 - Lineage is lost between related issues

- The entire process is fragmented, manual, and disconnected

Proposed Solution (Initial Design)

- Agentic multi-repository bug management system acting as a single unified interface ▶
- Core concept: One bug submission triggers automatic actions across all relevant systems ▶
- System capabilities: ▶ ▶
 - Multi-repository ticket creation based on affected components
 - Consolidated updates from different systems in one view
 - Federated search allowing engineers to access related information from a single place
 - Automated classification and intelligent routing
 - Complete lineage maintenance linking all tickets back to the original issue

Architecture Components Presented

- Customer-facing UI with bug submission form ▶
- JWT authentication for user authorization ▶
- Fast API gateway (Python web server) handling front-end and back-end requests ▶
- Kafka topic for queuing and decoupling: ▶ ▶
 - Maintains queue of incoming bugs
 - Prevents data loss if orchestrator or agents are down
 - Allows agents to process bugs at their own pace without being overwhelmed
- Orchestrator coordinating agents and ensuring system can resume after failures ▶
- Agent Pipeline with four sequential agents: ▶ ▶ ▶
 - **Intake Agent:** Cleans and normalizes input, generates unique case ID

- **Classification Agent:** Uses LLM to determine severity and identify affected components, with keyword-based fallback
- **Routing Agent:** Uses database lookups to determine which teams and platforms need involvement
- **CRUD Agent:** Creates actual tickets in destination systems, checks for duplicates to ensure idempotency
- PostgreSQL database with three tables: ▶ ▶ ▶ ▶ ▶
 - **Work Cases Table:** Tracks every pipeline step, enables system to resume after crashes
 - **CMDB Team Registry Table:** Maps components to teams and their tracking systems
 - **Work Items Table:** Prevents duplicate ticket creation and powers unified view
- Connector registry translating commands to specific platform connectors ▶
- Docker Compose for containerization and deployment ▶

Critical Feedback Received

- **Major Issue - Data Duplication:** Creating a parallel data system defeats the purpose of existing systems and is impractical ▶ ▶ ▶
 - Would require duplication of efforts and data storage costs (potentially doubling infrastructure costs) ▶
 - Not possible to replicate all data due to RBAC (Role-Based Access Control) and data sovereignty concerns ▶ ▶
 - Systems handling sensitive data must remain intact ▶
- **Incorrect Assumption - New Bug Portal:** The requirement is not to create a new unified bug submission interface ▶ ▶ ▶
 - Users should continue using their existing bug tracking systems ▶
 - The goal is to query and aggregate data, not replace existing workflows ▶ ▶

- **Revised Goal Clarification:** The system should: ▶ ▶ ▶
 - Query existing bug tracking systems without duplicating their data
 - Use modern agentic AI to gather information from discrete data sources
 - Present information in a unified, meaningful manner
 - Simplify and speed up bug triage by intelligently correlating information
 - Reduce turnaround time for bug triage process
- **Missing Component - UI Mockup:** Need to show end-user perspective to clarify requirements and identify bottlenecks ▶ ▶
 - Should show how related issues, customer case notes, user stories/epics, and bugs are displayed together ▶

Design Principles to Retain

- **Good Decoupling Approach:** The team's thinking about decoupling components is sound ▶ ▶
 - Kafka-based queuing to prevent system overload is a good pattern ▶
 - Design allows parts of the system to continue functioning even if other parts break ▶
- **Modular Architecture:** The decomposition and modularization approach is valuable ▶
- **Unified View Concept:** Creating a single interface where engineers can see information from multiple bug tracking platforms without navigating to each system separately is aligned with requirements ▶ ▶ ▶
 - The idea is to show information and reference original bugs in their respective systems ▶

Meeting Logistics

- Feedback can be shared in chat between monthly meetings ▶
- Additional meeting can be scheduled if design calls for more discussion ▶
- Team should iterate on design and share when ready for another review ▶